PROMPT COMPLEXITY CLASSIFIER

A Cost-Optimized LLM Routing System

Detailed Guidance Report

Prepared for final year project presentation

Prepared by: Nirmit Rampal
B.Tech CSE

This document explains the complete project plan, design, data flow, model training idea, evaluation logic, and deployment outline in simple and formal English.

Executive Summary

This project builds a lightweight machine learning router that decides whether a user prompt should be sent to a weak, intermediate, or strong language model. The main idea is to reduce API cost without making the system slow or complex. Instead of using embeddings, the project relies on simple text features and rule-based signals so that prediction stays fast and practical.

The project is designed as a real cost-saving layer between the user and the model provider. A prompt comes in, the router measures how difficult it appears, and the classifier predicts the minimum model tier that should be used. This makes the system useful in any application where many requests are simple, but some require deeper reasoning. In such cases, the router can save money on easy queries and still protect quality on harder ones.

The report describes the full project flow from problem statement to deployment. It includes the architecture, dataset design, labeling method, feature engineering strategy, model training plan, evaluation metrics, latency estimate, pseudocode, and future improvements. The goal is to give a complete guidance document that can be shown in a final year review, viva, or project demonstration.

Document Roadmap

Section	Topic
1	Introduction and motivation
2	Problem statement and objectives
3	System architecture
4	Dataset creation and labeling
5	Feature engineering without embeddings
6	Model training and evaluation
7	Graphs, diagrams, and analytics
8	Deployment and latency planning
9	Pseudocode and implementation plan
10	Risks, limitations, and future scope

The rest of the report follows this structure in order. Each section explains what will be done, why it is needed, and how it fits into the full routing system.

1. Introduction

Modern AI applications often use large language models for tasks such as answering questions, coding help, summarization, planning, and writing. Not every prompt needs the most powerful model. Many queries are short, direct, and simple enough for a cheaper model. However, applications usually send every request to one model tier by default, which increases cost and may not be efficient.

This project addresses that gap by building a prompt complexity classifier. The classifier estimates how much model strength is needed before the prompt is sent to an LLM. It acts like a gatekeeper. Instead of asking every model to do every job, the router chooses a model tier that is just strong enough for the task.

The project is intentionally designed without embeddings. This keeps the system light, easy to understand, and fast to run. The focus is on handcrafted features, keyword signals, and XGBoost classification. This makes the project appropriate for academic presentation because the whole flow is easy to explain, test, and improve step by step.

1.1 Motivation

- LLM costs rise quickly when every request uses a strong model.
- Many prompts are simple and do not need deep reasoning.
- A routing layer can lower cost while keeping quality acceptable.
- A lightweight ML model is easier to deploy than a heavy neural model.

2. Problem Statement

The problem is to predict the minimum model tier required for a given prompt. The system must classify the prompt into one of three categories: weak, intermediate, or strong. The classification should be accurate enough to avoid under-routing hard prompts and over-routing easy prompts.

If the system routes a difficult prompt to a weak model, the output quality may fail. If it routes a simple prompt to a strong model, unnecessary money is spent. Therefore, the routing decision must balance cost and quality. This is the central tradeoff of the project.

The challenge becomes even more interesting because the input is not a structured record like a table of numbers. The input is raw text. That means the solution must extract useful signals from language itself, without relying on embeddings or deep semantic vectors.

2.1 Core Questions

1. How can prompt difficulty be estimated from text alone?
2. What features are simple enough to compute quickly but strong enough to help the classifier?
3. How can synthetic labels be created in a way that supports training?
4. How can the system prove cost savings without sacrificing too much quality?

3. Project Objectives

Objective	Description
Reduce API cost	Send only the necessary prompts to stronger model tiers.
Keep latency low	Use a fast classifier and avoid embedding overhead.

Create a usable dataset	Generate prompt samples and label them by minimum required model.
Build an explainable system	Use visible rules and simple features that can be demonstrated easily.
Support deployment	Make the model usable as an API routing component.

These objectives shape the whole solution. The project is not only about achieving a good accuracy score. It is about building a routing strategy that has a real operational purpose. The classifier must help a system spend less money and still behave sensibly under normal use.

4. Scope of the Project

The scope covers prompt classification, not LLM generation itself. The project does not attempt to improve the model outputs directly. Instead, it decides which model should be used. This keeps the project focused and manageable within a final year timeline.

The solution includes dataset preparation, feature extraction, model training, testing, and deployment planning. It does not depend on embeddings, vector search, retrieval pipelines, or complex neural architectures. That choice is intentional because the project aims to stay lightweight and easy to showcase.

5. System Architecture

The architecture is built as a small decision pipeline. A prompt enters the system, quick rule checks are applied, text features are extracted, and the XGBoost model predicts the class. A confidence check can then decide whether to trust the prediction or use a safer fallback. This flow keeps the system simple and production-friendly.

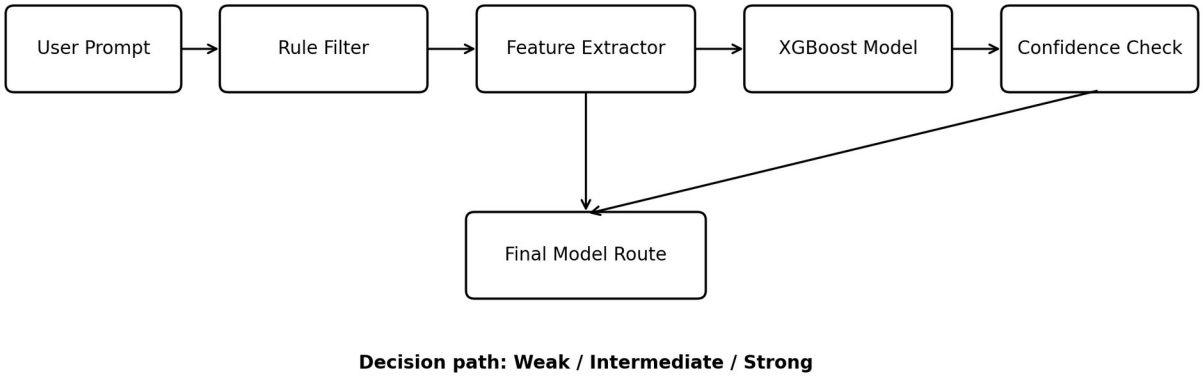

Figure 1: System architecture of the prompt routing classifier.

The architecture is intentionally modular. Each part has one job. The prompt parser handles text cleanup, the feature extractor computes numeric signals, the classifier predicts the class, and the confidence layer prevents weak decisions when the model is uncertain. This modular design is useful for debugging and later upgrades.

5.1 Component Description

Component	Role
Input layer	Receives the raw prompt from the user or application.
Rule-based filter	Handles obvious cases using keywords or simple overrides.
Feature extractor	Converts text into numeric features.
Classifier	Predicts weak, intermediate, or strong label.
Confidence layer	Checks whether the prediction is stable enough to trust.
Output router	Returns the selected model tier to the calling system.

6. Dataset Design and Data Flow

The dataset is the most important part of the whole project. A classifier is only as good as the examples it learns from. Because the project is meant to route prompts by difficulty, the dataset must include a wide range of prompt types. It must also reflect the minimum model needed, not just whether a strong model gives a polished answer.

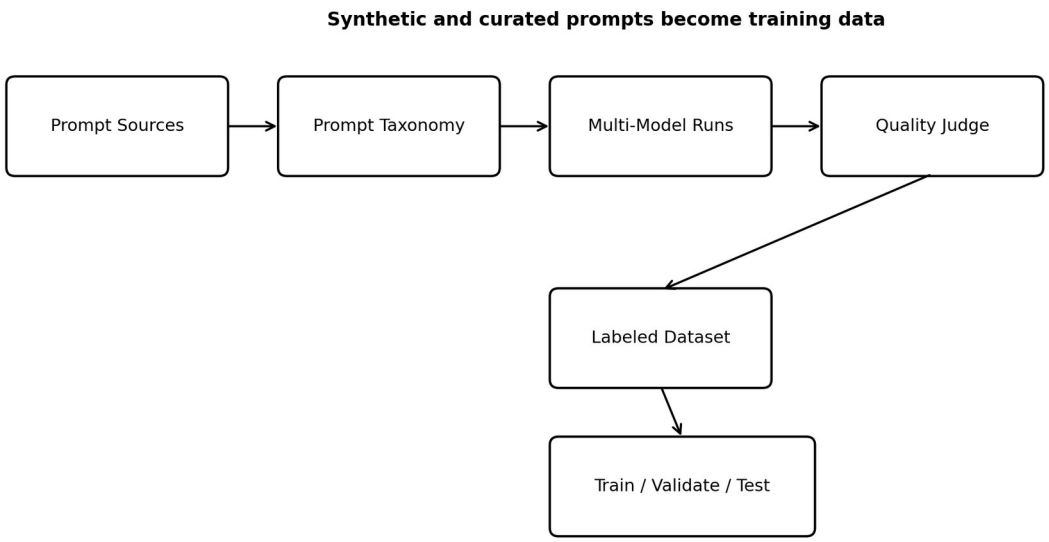

Figure 2: Data generation and labeling pipeline.

6.1 Prompt Categories

Category	What it covers	Example
Simple factual	Direct questions with short answers	What is the capital of Japan?
Reasoning	Logic, explanation, and analysis	Explain why the sky appears blue.
Coding	Programming tasks and algorithm requests	Write a BFS function in C++.
Debugging	Fixing broken code or errors	Find the bug in this loop.

Creative writing	Stories, poems, and style tasks	Write a short poem about rain.
Multi-step planning	Tasks with several steps or decisions	Plan a launch strategy for a startup.

The categories are chosen because they naturally differ in difficulty. A factual question may be easy for a weak model, while a multi-step design task may require a stronger model. This diversity helps the classifier learn useful patterns rather than memorize one type of prompt.

6.2 Source of Prompts

- Synthetic generation using carefully designed prompt templates.
- Curated prompts from public sources such as coding, Q&A, and discussion datasets.
- Manual prompts created to represent common real-world use cases.
- Balanced sampling so that one prompt type does not dominate the dataset.

7. Labeling Strategy

The labeling method defines what the model learns. For this project, the label should represent the minimum model tier that can handle the prompt well enough. This is more useful than asking which model simply sounds best, because the goal is cost reduction, not style preference.

The project can label data by running the same prompt through weak, intermediate, and strong models. The outputs are then checked against a quality rule. If the weak model is acceptable, the label is weak. If weak fails but intermediate succeeds, the label is intermediate. If only strong is good enough, the label is strong.

This approach is practical because it directly matches the routing goal. The classifier will not learn to admire the strongest answer by default. Instead, it will learn the cheapest acceptable answer tier. That makes the dataset more aligned with the real use case.

7.1 Label Assignment Rules

Observed result	Assigned label
Weak model output is acceptable	Weak
Weak fails but intermediate is acceptable	Intermediate
Only strong is acceptable	Strong

7.2 Quality Judging Rules

- Correctness: the answer should be factually or logically valid.
- Completeness: the answer should cover the required request.
- Clarity: the answer should be understandable and not confused.
- Safety of routing: the chosen tier should not be too weak for the task.

8. Feature Engineering Without Embeddings

The project deliberately avoids embeddings. That means the model must rely on visible text signals rather than dense semantic vectors. This is a strength for an academic demo because the features are easy to explain, easy to inspect, and fast to compute. It also keeps the latency of the router low.

The features should not be random. They should reflect properties that often correlate with prompt complexity. A short direct question is usually simpler than a long multi-part planning request. A prompt

containing words like explain, design, compare, debug, or optimize may need more reasoning than a plain definition request.

8.1 Basic Text Features

- Character count of the prompt.
- Word count and sentence count.
- Average word length as a rough complexity signal.
- Number of punctuation marks and question marks.
- Presence of code-like symbols such as brackets, braces, and semicolons.

8.2 Keyword and Rule Features

- Reasoning words: explain, analyze, compare, justify, infer.
- Coding words: code, function, implement, class, bug, error.
- Planning words: design, strategy, roadmap, architecture, system.
- Creative words: story, poem, rewrite, imagine, style.
- Multi-step words: steps, process, workflow, phase, sequence.

8.3 Why These Features Are Enough for a First Version

A first version does not need to understand language in a deep neural way. It only needs to separate very simple prompts from clearly complex prompts. The chosen features are enough to give the classifier useful structure. In practice, this can already produce a strong baseline if the dataset is well made.

The features also support explainability. If the model predicts strong for a prompt, the developer can point to visible signals such as long length, multi-step wording, and planning keywords. This makes the project easier to defend in a viva because the decision is not a black box.

9. Model Training Plan

The chosen classifier is XGBoost. It is a good fit because it handles tabular features well, trains quickly, and often performs strongly on structured data. Since the project features are numeric and categorical flags, this model is a natural choice.

Training is a supervised learning task. Each prompt becomes one row in the dataset. The engineered features become the input columns. The label becomes the target class. After training, the model learns combinations of signals that indicate weak, intermediate, or strong routing needs.

9.1 Training Process

Step	Description
Split data	Use train, validation, and test splits.
Fit model	Train XGBoost on feature vectors and labels.
Tune parameters	Adjust depth, learning rate, and number of trees.
Validate	Check F1 score and class-wise errors.
Finalize	Lock the best version for deployment.

9.2 Why XGBoost Fits This Project

- It is fast at prediction time.
- It works well with hand-crafted features.
- It usually performs better than a simple threshold rule.

- It is easier to explain than a deep neural classifier.

10. Evaluation and Graphs

The project should not be judged only by accuracy. Because the system exists to save money, the evaluation must include cost reduction and routing quality. The best project report will show both the algorithmic performance and the practical benefit.

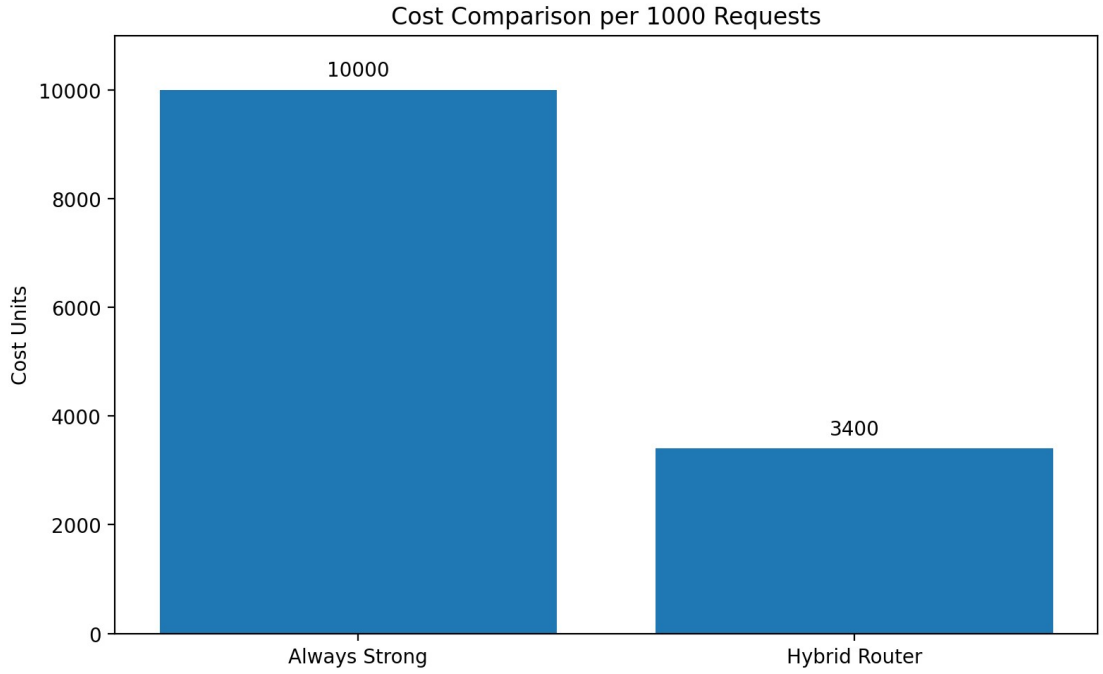

Figure 3: Estimated cost comparison per 1000 requests.

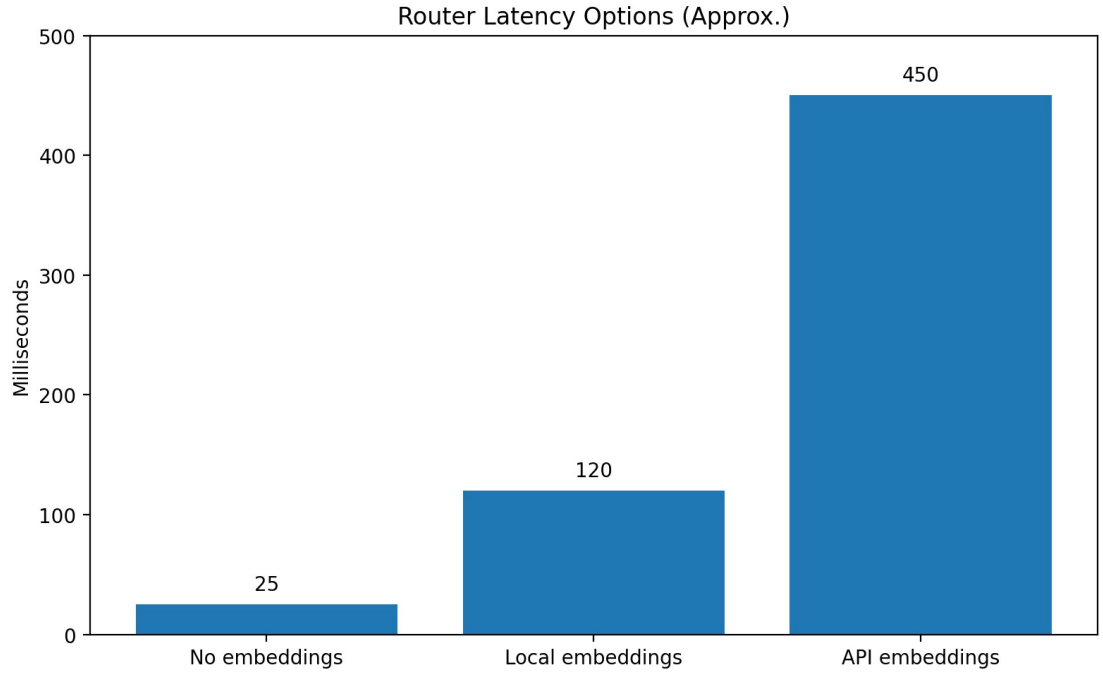

Figure 4: Approximate router latency under three feature setups.

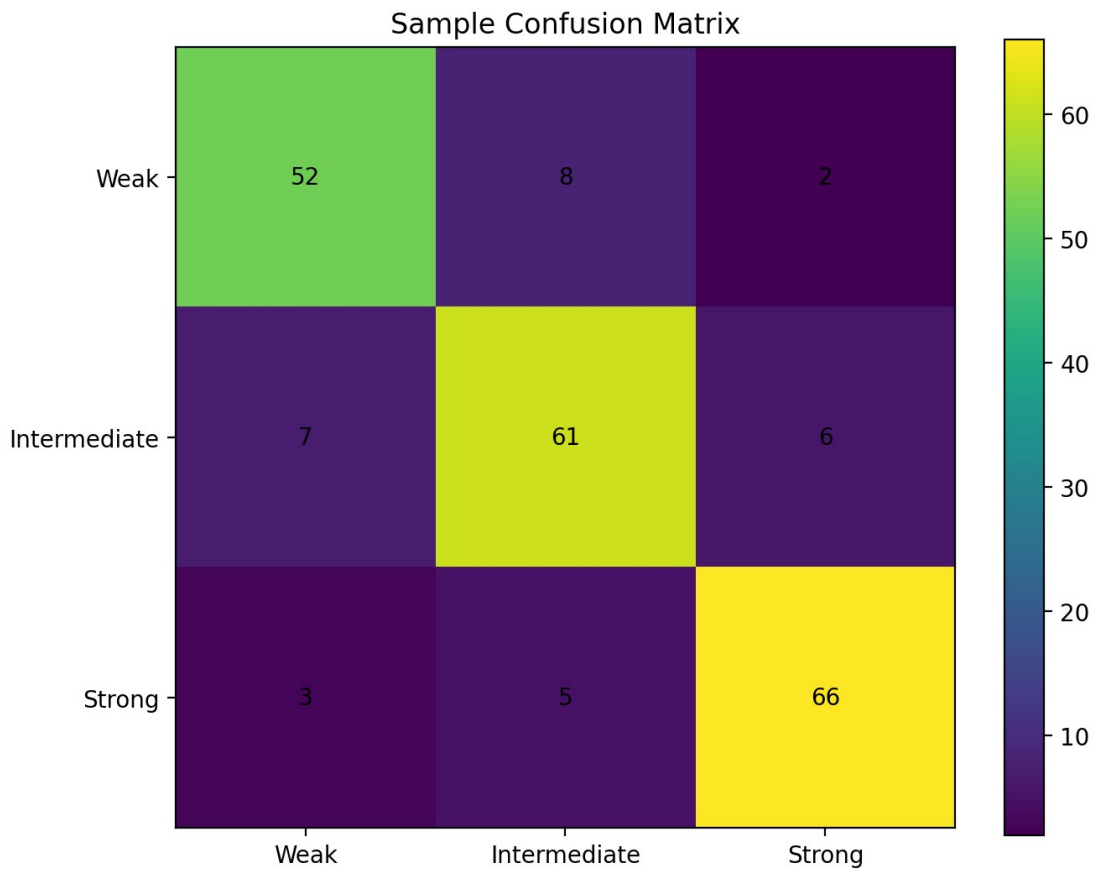

Figure 5: Sample confusion matrix for three routing classes.

10.1 Metrics to Report

Metric	Why it matters
Accuracy	Shows the overall percentage of correct class predictions.
Precision	Shows how often a predicted class was right.
Recall	Shows how well the model catches each class.
F1 score	Useful when classes are not perfectly balanced.
Confusion matrix	Shows where the model confuses one class with another.
Cost saved	Shows the real business value of the router.

10.2 Expected Result Pattern

The model should ideally be conservative when the prompt looks difficult and cheaper when the prompt clearly looks simple. Some mistakes are acceptable, but the system should avoid sending hard prompts to weak models too often. A small number of over-escalations is better than a dangerous under-escalation.

11. Runtime Flow

The following sequence diagram shows what happens when a request enters the system. This view is useful because it explains the order of events, not only the internal architecture.

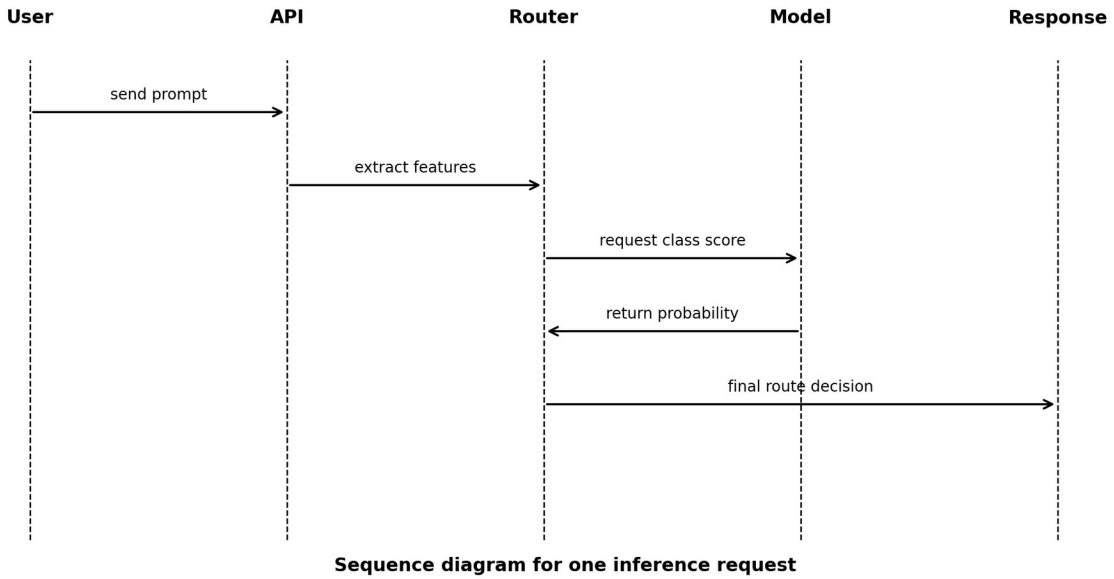

Figure 6: Sequence diagram for one routing request.

12. Deployment View

The deployment view shows how the router can sit inside a real application. The client sends the prompt, the API handles request processing, the feature engine builds the input, the classifier predicts the class, and the result is logged for later analysis.

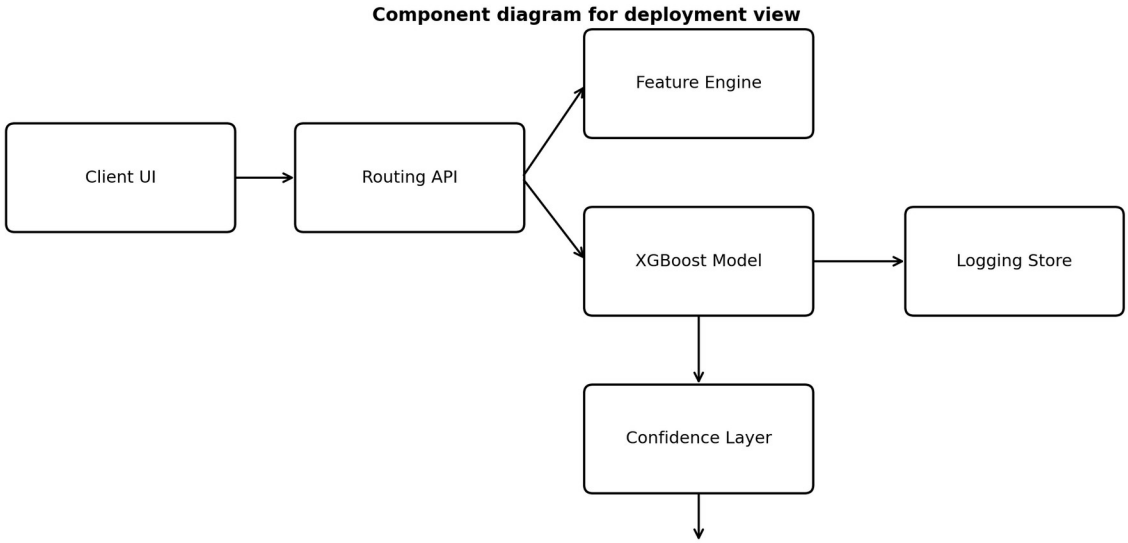

Figure 7: Deployment-oriented component diagram.

12.1 Logging and Monitoring

- Store prompt length, predicted class, confidence, and final decision.

- Track how often the router escalates to a stronger model.
- Monitor confusion cases to improve the dataset later.
- Use logs to build a feedback loop for future retraining.

13. Pseudocode

The pseudocode below describes the logic in a simple way. It is intentionally not written as full code so that the report stays conceptual and easy to present.

```
Algorithm 1: Prompt routing
Input: raw prompt text
Output: weak, intermediate, or strong model tier

1. Receive the prompt
2. Clean the text and normalize spacing
3. Apply simple rule checks for obvious cases
4. Extract basic text features
5. Convert features into a numeric vector
6. Pass the vector to the XGBoost classifier
7. Obtain class probabilities
8. If confidence is too low, move to a safer class
9. Return the final model tier
```

13.1 Training Pseudocode

```
1. Build a prompt dataset
2. Label each prompt by minimum required model
3. Extract handcrafted features
4. Split the data into train and test sets
5. Train XGBoost on the training set
6. Measure performance on the test set
7. Save the best model
```

14. Latency and Complexity Analysis

Because embeddings are not used, the router remains lightweight. Feature extraction based on simple text statistics is fast, and XGBoost prediction is also fast. This means the routing layer can work as a small middleware component without making the application feel heavy.

In normal conditions, the expected router latency is low enough to be acceptable for interactive applications. The main source of delay would be external system overhead, not the classifier itself. That is exactly why this design is practical.

Stage	Approximate time
Text cleanup and feature extraction	5 to 20 ms
XGBoost prediction	1 to 5 ms
Total router latency	10 to 30 ms in a simple setup

14.1 Complexity View

- The feature extractor runs in time proportional to prompt length.
- XGBoost inference is very fast for a small feature vector.
- The router does not need GPU acceleration for the first version.

15. Risks, Limitations, and Mitigation

No classifier is perfect. A prompt can look simple while actually needing deep reasoning. Another prompt can look long but still be easy. The system therefore needs good feature design and a sensible fallback policy. A confidence threshold is helpful here because uncertain predictions should not be trusted blindly.

Synthetic labels are another risk. If the labeling model is biased, the dataset may also become biased. To reduce this problem, the prompt set should be diverse and the quality checks should be defined carefully. It also helps to test the router on unseen prompts that were not used during generation.

Risk	Mitigation
Prompt length bias	Use multiple features, not length alone.
Synthetic label noise	Apply clear quality criteria and human review when possible.
Under-routing hard prompts	Use confidence fallback to safer classes.
Class imbalance	Balance prompt categories in the dataset.

16. Future Enhancements

Once the first version works, the system can be improved step by step. For example, a future version can include embeddings, but that is not necessary for the first project. Another upgrade could add active learning, where uncertain prompts are sent for manual review and then used for retraining.

The router can also be extended to more than three tiers. Some systems may want separate classes for ultra-cheap, cheap, medium, strong, and premium models. The same design can scale to that case with larger training data and more class labels.

- Add more routing tiers if needed.
- Use feedback logs to improve labels over time.
- Add dashboards for cost and quality tracking.
- Integrate with a real model gateway or proxy service.

17. Conclusion

This project proposes a practical and explainable way to reduce LLM usage cost by classifying prompt difficulty before the model call happens. The system is intentionally simple: it uses handcrafted features, avoids embeddings, and relies on XGBoost for fast routing. This makes it easy to explain, quick to run, and suitable for a final year showcase.

The strongest part of the project is that it solves a real operational problem. It does not try to replace the language model. It decides which model should be used. That is a smart and useful layer in modern AI systems. With good data, clear labeling, and careful evaluation, the project can become a very solid demonstration of applied machine learning and cost-aware system design.

Appendix A: Suggested Prompt Samples

Type	Sample prompt
Simple factual	What is photosynthesis?
Coding	Implement binary search in C++.
Debugging	Why is this loop not terminating?
Reasoning	Compare supervised and unsupervised learning.
Planning	Design a small app launch strategy.
Creative	Write a short story about a robot and a clock.

Appendix B: Suggested Feature List

Feature group	Examples
Length features	Character count, word count, sentence count
Shape features	Average word length, punctuation count
Keyword flags	Reasoning, coding, planning, creative
Structure flags	Code symbols, question marks, numbers
Heuristic score	Combined rough complexity score

18. Step-by-Step Execution Plan

This section explains the full project in the exact order in which the system will be built and tested. It is useful for presentation because it makes the whole workflow easy to explain from start to finish.

- Step 1: collect prompt samples from different task categories.
- Step 2: label each prompt using the minimum model tier that gives an acceptable answer.
- Step 3: convert the prompt into text features such as length, punctuation, and keywords.
- Step 4: train the XGBoost classifier on the labeled feature table.
- Step 5: test the classifier on unseen prompts and record accuracy and confusion cases.
- Step 6: set a confidence threshold for uncertain predictions.
- Step 7: deploy the router as a small API service.
- Step 8: log each decision for later analysis and retraining.

The project should be developed in this order because each step depends on the previous one. Data quality comes first. Model quality comes second. Deployment comes only after the router behaves well on test prompts.

18.1 What Happens at Runtime

At runtime, the user submits a prompt. The router cleans the prompt, extracts a compact feature set, and runs the classifier. The result is a label and a confidence score. If the score is strong enough, the system returns that label immediately. If the score is weak, the system upgrades the decision to a safer class.

This is the main logic that makes the project practically useful. A router that is too strict will waste money. A router that is too risky will hurt output quality. The runtime plan keeps both concerns in balance.

19. Validation and Testing Plan

Testing is important because the project deals with routing decisions, not just raw classification accuracy. A good test plan checks both machine learning metrics and practical behavior. The system should be tested on prompt types that were seen in training as well as new prompts that were not.

Test type	Purpose
Unit test	Check prompt cleanup, feature extraction, and label mapping.
Model test	Check accuracy, F1 score, and confusion matrix.
Stress test	Measure response time across many prompts.
Ablation test	See what happens if a feature group is removed.
Scenario test	Use real-world prompts and judge the routing choice.

An ablation test is especially useful here. For example, the project can be evaluated with and without keyword features. If performance drops sharply after removing those features, it proves that the features are useful. This makes the report more convincing because it shows why the design choices matter.

20. Sample Use Cases

A practical project becomes more believable when it is tied to real use cases. The router can be used in any product where requests vary in difficulty. The same idea works for chatbots, support systems, content tools, and coding assistants.

Use case	Why routing helps
Customer support chatbot	Simple FAQ answers can use a cheaper model.
Coding assistant	Easy syntax questions stay cheap, while complex debugging escalates.
Content tool	Basic rewriting is cheap, but detailed planning uses a stronger model.
Internal enterprise assistant	Many internal questions are simple and repetitive.

This makes the project easy to justify in a viva. It is not a toy classification problem. It is a decision layer that can save money in real systems with mixed query difficulty.

21. Project Deliverables

The final project should include the report, the dataset design, the trained model, the evaluation results, the diagrams, and the deployment plan. A clean final year submission should make it obvious what was built, how it works, and what the results mean.

- Detailed report document.
- Dataset design and labeling explanation.
- Feature list and model training plan.
- Evaluation metrics, graphs, and confusion matrix.
- Deployment concept with runtime flow.
- Pseudocode for training and routing.

22. Presentation Notes

When presenting this project, the explanation should stay simple. Start with the cost problem, then explain why all prompts should not be sent to the strongest model. After that, show the architecture, data labeling, and feature engineering. Finally, show the graphs that prove the router can save cost while staying fast.

A good viva answer should emphasize that the router is a practical optimization layer. It does not replace the LLM. It chooses the right LLM tier. That is the key idea, and it should be repeated clearly when presenting the project.